

11-02 — Relations

Section	11 — Entities, Relations, and CRUD with TypeORM
Subtopic	11-02 — Relations
Est. Duration	2 – 2.5 hours
Type	Theory Practical TypeScript
Topics covered	<code>@OneToOne</code> , <code>@OneToMany</code> , <code>@ManyToOne</code> , <code>@ManyToMany</code> , <code>@JoinColumn</code> , <code>@JoinTable</code> , inverse sides, cascade options, eager vs lazy loading

Learning Objectives — This Subtopic

- Identify the correct relation type (One-to-One, One-to-Many, Many-to-Many) for a given domain model and articulate why.
- Apply `@OneToOne` , `@OneToMany` , `@ManyToOne` , and `@ManyToMany` decorators with bidirectional inverse sides.
- Explain the role of `@JoinColumn` and `@JoinTable` and which side of a relation owns each.
- Configure cascade options to control how operations on an owner entity propagate to related entities.
- Distinguish between eager and lazy loading and choose the appropriate strategy for a given query pattern.
- Recognise and avoid the N+1 query problem by using TypeORM's `relations` option or `QueryBuilder` joins.

1. Relations in a Relational Database

Relations describe how rows in one table connect to rows in another. In SQL these connections are implemented with **foreign keys** — a column in one table that stores the primary key of a row in another. TypeORM makes foreign keys first-class citizens of your TypeScript model: instead of writing `userId` `INTEGER REFERENCES users(id)` in SQL, you decorate a property on your entity class and TypeORM generates the constraint automatically.

TypeORM supports four relation types, each corresponding to a real-world cardinality:

Relation	Meaning	Example
@OneToOne	One row in A → exactly one row in B	User ↔ Profile
@OneToMany	One row in A → many rows in B	Category → Products
@ManyToOne	Many rows in A → one row in B	Products → Category
@ManyToMany	Many rows in A ↔ many rows in B	Products ↔ Tags

Analogy: Think of a relation as a *named reference* between two filing cabinets. A One-to-One relation is like giving one employee exactly one locker with their name on it. A One-to-Many is like one department shelf containing many employee folders. Many-to-Many is like a shared interest group — one employee can belong to many groups, and one group can contain many employees; you need a sign-up sheet (a junction table) to track the membership.

Prerequisites from 11-01: The entity classes and DataSource configuration used throughout this subtopic build directly on the `Product`, `Category`, and `Country` entities from 11-01. Ensure `synchronize: true` is active in your development DataSource, and that `experimentalDecorators` and `emitDecoratorMetadata` are enabled in `tsconfig.json`.

2. One-to-One Relations — @OneToOne

A One-to-One relation means each row in table A is linked to at most one row in table B, and vice versa. The foreign key can live on either side; the side that holds the foreign key column is the **owning side**, marked with `@JoinColumn()`.



user holds a FK → profile.id (or profile holds a FK → user.id)

2.1 Owning Side

STAGE 1 Create the `Profile` entity — the "target" of the relation.

src/entity/Profile.ts

```
import { Entity, PrimaryGeneratedColumn, Column } from "typeorm";

@Entity("profiles")
export class Profile {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({ nullable: true, length: 160 })
  bio: string | null;

  @Column({ nullable: true, length: 80 })
  website: string | null;
}
```

Output:

No runtime output – entity class definition only.

STAGE 2 Add the relation to the `User` entity. The `User` is the owning side because it holds the foreign key column (`profileId`).

src/entity/User.ts

```
import {
  Entity, PrimaryGeneratedColumn, Column,
  OneToOne, JoinColumn,
  CreateDateColumn,
} from "typeorm";
import { Profile } from "../Profile";

@Entity("users")
export class User {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({ unique: true, length: 100 })
  email: string;

  @Column({ length: 60 })
  passwordHash: string;

  // This side owns the foreign key - @JoinColumn goes here.
  @OneToOne(() => Profile, { nullable: true, cascade: true })
  @JoinColumn() // creates column "profileId" on the users table
  profile: Profile | null;

  @CreateDateColumn()
  createdAt: Date;
}
```

Output:

Generated SQL (SQLite):

```
CREATE TABLE users (
  "id"          integer      PRIMARY KEY AUTOINCREMENT NOT NULL,
  "email"       varchar(100) NOT NULL UNIQUE,
  "passwordHash" varchar(60) NOT NULL,
  "profileId"   integer,
  "createdAt"   datetime     NOT NULL DEFAULT (datetime('now')),
  FOREIGN KEY ("profileId") REFERENCES "profiles" ("id") ON DELETE NO ACTION
);
```

A few things to note about this code:

- `@JoinColumn()` is mandatory on exactly one side of a One-to-One relation — the owning side. It tells TypeORM which table gets the foreign key column. If you omit it, TypeORM will throw an error.

- `() => Profile` is a *type function*. TypeORM uses a function (rather than a direct class reference) to avoid circular dependency issues between entity files that mutually import each other.
- `cascade: true` is shorthand for enabling all cascade operations (insert, update, remove). When you save a `User` that has a `profile` property set, TypeORM will automatically save the `Profile` as well without a separate `save()` call.
- `nullable: true` on the relation allows the foreign key column to be NULL — i.e., a user may exist without a profile.

2.2 Bidirectional Inverse Side

STAGE 3 A relation can be declared on both entities (bidirectional), allowing you to navigate from either side. The non-owning side uses the second argument of the decorator to point back to the owning side's property.

src/entity/Profile.ts (updated)

```
import { Entity, PrimaryGeneratedColumn, Column, OneToOne } from "typeorm";
import { User } from "../User";

@Entity("profiles")
export class Profile {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({ nullable: true, length: 160 })
  bio: string | null;

  @Column({ nullable: true, length: 80 })
  website: string | null;

  // Inverse side – no @JoinColumn here
  @OneToOne(() => User, (user) => user.profile)
  user: User;
}
```

Output:

No additional SQL is generated. The inverse side is purely navigational – it adds no column or constraint to the profiles table.

Key Insight: Bidirectionality is a TypeORM concept, not a database concept. The database schema is identical whether you declare the inverse side or not. The inverse side simply tells TypeORM how to populate the reverse property when you query with `relations` or join in QueryBuilder (Section 12).

2.3 Custom Foreign Key Column Name

By default TypeORM names the FK column `propertyName + "Id"` (e.g., `profileId`). Override it with `@JoinColumn({ name: "profile_id" })`:

src/entity/User.ts (excerpt)

```
@OneToOne(() => Profile, (profile) => profile.user, { nullable: true, cascade: true })
@JoinColumn({ name: "profile_id" }) // column in DB is "profile_id" not "profileId"
profile: Profile | null;
```

Output:

Generated column name in `users`: `profile_id` (instead of `profileId`)

3. One-to-Many and Many-to-One — `@OneToMany` / `@ManyToOne`

One-to-Many and Many-to-One are two sides of the same relation. One `Category` can contain many `Product`s; each `Product` belongs to exactly one `Category`. The foreign key always lives on the *Many* side — in this case the `products` table holds a `categoryId` column.



The **Many side** (`Product`) uses `@ManyToOne` and is the owning side (holds the FK). The **One side** (`Category`) uses `@OneToMany` and is the inverse side. Unlike One-to-One, you do *not* use `@JoinColumn` here — TypeORM infers the FK column from the `@ManyToOne` decorator alone.

3.1 Defining Both Sides

STAGE 1 The `Category` entity (One side).

src/entity/Category.ts

```
import { Entity, PrimaryGeneratedColumn, Column, OneToMany } from "typeorm";
import { Product } from "../Product";

@Entity("categories")
export class Category {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({ unique: true, length: 80 })
  name: string;

  @Column({ unique: true, length: 80 })
  slug: string;

  // Inverse side – does NOT add a column to the categories table
  @OneToMany(() => Product, (product) => product.category)
  products: Product[];
}
```

Output:

Generated SQL:

```
CREATE TABLE categories (
  "id" integer PRIMARY KEY AUTOINCREMENT NOT NULL,
  "name" varchar(80) NOT NULL UNIQUE,
  "slug" varchar(80) NOT NULL UNIQUE
);
-- No FK column here. FK lives in the products table.
```

STAGE 2 The `Product` entity (Many side) — update it to include the relation.

src/entity/Product.ts (updated excerpt)

```
import {
  Entity, PrimaryGeneratedColumn, Column,
  ManyToOne, CreateDateColumn, UpdateDateColumn,
} from "typeorm";
import { Category } from "../Category";
import { ProductStatus } from "../ProductStatus"; // enum moved to its own file for clarity

@Entity("products")
export class Product {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({ length: 120, unique: true })
  name: string;

  @Column("decimal", { precision: 10, scale: 2 })
  price: number;

  @Column({ type: "enum", enum: ProductStatus, default: ProductStatus.Draft })
  status: ProductStatus;

  @Column({ nullable: true, type: "text" })
  description: string | null;

  @Column({ default: 0, type: "int" })
  stockCount: number;

  // Owning side – this entity holds the FK column "categoryId"
  @ManyToOne(() => Category, (category) => category.products, {
    nullable: true,
    onDelete: "SET NULL", // if the category is deleted, set categoryId to NULL
  })
  category: Category | null;

  @CreateDateColumn()
  createdAt: Date;

  @UpdateDateColumn()
  updatedAt: Date;
}
```

Output:

Generated SQL:

```
CREATE TABLE products (
  "id" integer PRIMARY KEY AUTOINCREMENT NOT NULL,
  "name" varchar(120) NOT NULL UNIQUE,
  "price" decimal(10,2) NOT NULL,
  "status" varchar NOT NULL DEFAULT 'draft',
  "description" text,
  "stockCount" integer NOT NULL DEFAULT (0),
  "categoryId" integer,
  "createdAt" datetime NOT NULL DEFAULT (datetime('now')),
  "updatedAt" datetime NOT NULL DEFAULT (datetime('now')),
  FOREIGN KEY ("categoryId") REFERENCES "categories" ("id") ON DELETE SET NULL
);
```

3.2 The `onDelete` Option

The `onDelete` option in `@ManyToOne` controls what the database does to the child rows when the parent is deleted. The three most used values are:

Value	Behaviour	Use When
<code>"NO ACTION"</code> (default)	Database raises an error if you delete a referenced parent	You want to enforce referential integrity strictly
<code>"CASCADE"</code>	Deleting a parent also deletes all its children	Children are meaningless without the parent (e.g. order items when order is deleted)
<code>"SET NULL"</code>	Deleting a parent sets the FK column to NULL on children	Children can exist independently (e.g. products when a category is deleted)

Important — `cascade` vs `onDelete` : These are distinct and commonly confused. `cascade: true` in the TypeORM decorator options tells TypeORM's JavaScript layer to propagate `save()` / `remove()` calls through your code. `onDelete` is a database-level constraint that the RDBMS enforces regardless of how the delete happens. Use both where appropriate — they complement each other.

4. Many-to-Many Relations — @ManyToMany

A Many-to-Many relation cannot be expressed with a single foreign key column because one row in A links to multiple rows in B, and those rows in B each link back to multiple rows in A. The standard SQL solution is a **junction table** (also called a join table or pivot table) — a third table whose sole purpose is to record which A rows are paired with which B rows.

Product

∞ — junction — ∞

Tag

product_tags table: (productId FK, tagId FK)

TypeORM generates and manages the junction table for you — you never create it directly. You control it through `@JoinTable()` on the **owning side** of the relation.

4.1 Defining the Relation

STAGE 1 Create the `Tag` entity.

src/entity/Tag.ts

```
import { Entity, PrimaryGeneratedColumn, Column, ManyToMany } from "typeorm";
import { Product } from "../Product";

@Entity("tags")
export class Tag {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({ unique: true, length: 50 })
  name: string;

  // Inverse side – no @JoinTable here
  @ManyToMany(() => Product, (product) => product.tags)
  products: Product[];
}
```

Output:

Generated SQL:

```
CREATE TABLE tags (  
  "id" integer PRIMARY KEY AUTOINCREMENT NOT NULL,  
  "name" varchar(50) NOT NULL UNIQUE  
);  
-- No FK columns here. The junction table is created separately.
```

STAGE 2 Add `@ManyToMany` and `@JoinTable` to the `Product` entity. The owning side is the one that carries `@JoinTable()`.

src/entity/Product.ts (updated excerpt)

```
import {  
  Entity, PrimaryGeneratedColumn, Column,  
 ManyToOne, ManyToMany, JoinTable,  
  CreateDateColumn, UpdateDateColumn,  
} from "typeorm";  
import { Category } from "../Category";  
import { Tag } from "../Tag";  
import { ProductStatus } from "../ProductStatus";  
  
@Entity("products")  
export class Product {  
  @PrimaryGeneratedColumn()  
  id: number;  
  
  @Column({ length: 120, unique: true })  
  name: string;  
  
  @Column("decimal", { precision: 10, scale: 2 })  
  price: number;  
  
  @Column({ type: "enum", enum: ProductStatus, default: ProductStatus.Draft })  
  status: ProductStatus;  
  
  @Column({ nullable: true, type: "text" })  
  description: string | null;  
  
  @Column({ default: 0, type: "int" })  
  stockCount: number;  
  
  @ManyToOne(() => Category, (category) => category.products, {  
    nullable: true,  
    onDelete: "SET NULL",  
  })  
  category: Category | null;
```

```
// Owning side of Many-to-Many – @JoinTable goes here
@ManyToMany(() => Tag, (tag) => tag.products, { cascade: true })
@JoinTable({
  name: "product_tags", // explicit junction table name
  joinColumn: { name: "productId", referencedColumnName: "id" },
  inverseJoinColumn: { name: "tagId", referencedColumnName: "id" },
})
tags: Tag[];

@CreateDateColumn()
createdAt: Date;

@UpdateDateColumn()
updatedAt: Date;
}
```

Output:

```
Generated SQL (junction table):
CREATE TABLE product_tags (
  "productId" integer NOT NULL,
  "tagId" integer NOT NULL,
  PRIMARY KEY ("productId", "tagId"),
  FOREIGN KEY ("productId") REFERENCES "products" ("id") ON DELETE CASCADE ON UPDATE CASCADE,
  FOREIGN KEY ("tagId") REFERENCES "tags" ("id") ON DELETE CASCADE ON UPDATE CASCADE
);
```

TypeORM creates the junction table entirely from the `@JoinTable` configuration. The composite primary key ensures a product-tag pairing is unique. Both FKs use `ON DELETE CASCADE` by default so that junction rows are cleaned up when either product or tag is deleted.

4.2 @JoinTable Without Custom Names

Naming the junction table and its columns explicitly (as above) is good practice — it gives you predictable column names for raw SQL queries, migrations, and database tooling. However, TypeORM will generate defaults if you omit the options:

```
Minimal @JoinTable (no options)
```

```
@ManyToMany(() => Tag, (tag) => tag.products)
@JoinTable() // TypeORM generates: table "product_tag", columns "productId_id" and "tagId_id"
tags: Tag[];
```

Output:

Junction table auto-name: `product_tag` (entityName_entityName, lowercase)
FK columns: `productId_id`, `tagId_id` (TypeORM default naming, less readable)

Key Insight: Always provide explicit names to `@JoinTable`. The auto-generated names (especially the double `_id` suffix) are unintuitive and become a maintenance burden the moment you need to write raw SQL or a migration.

5. Cascade Options

The `cascade` option in a relation decorator controls whether TypeORM automatically propagates persistence operations from the owning entity to the related entity — without requiring you to call `save()` or `remove()` separately on the related entity.

`cascade` accepts either `true` (enables all operations) or an array of specific operations:

src/entity/User.ts (excerpt)

```
@OneToOne(() => Profile, { cascade: ["insert", "update"] })
@JoinColumn()
profile: Profile | null;
```

Output:

With `cascade: ["insert", "update"]`:

- Saving a User with a new Profile → TypeORM INSERTs the Profile automatically.
- Saving a User with a modified Profile → TypeORM UPDATEs the Profile automatically.
- Removing a User does NOT cascade – you must remove the Profile separately.

Cascade value	What TypeORM propagates automatically
<code>"insert"</code>	New related entities are saved on owner INSERT
<code>"update"</code>	Modified related entities are saved on owner UPDATE
<code>"remove"</code>	Related entities are deleted when owner is removed
<code>"soft-remove"</code>	Related entities are soft-deleted when owner is soft-deleted
<code>true</code>	All of the above

Warning — cascade `"remove"` : Enabling cascade remove means that deleting a parent entity automatically deletes all its related children through TypeORM. Use this cautiously — it is easy to accidentally delete large volumes of data. Prefer database-level `ON DELETE CASCADE` (via `onDelete: "CASCADE"`) for many-to-one relations; reserve TypeORM cascade remove for closely coupled value objects like a user → profile pair.

6. Eager vs Lazy Loading

By default, when you load an entity, TypeORM does *not* automatically load its related entities. You must explicitly request them. This is **lazy-by-default** behaviour — not to be confused with TypeORM's opt-in *lazy loading* feature. You control how related data is loaded through three mechanisms:

- **Eager loading** — declared on the entity; always loaded.
- Explicit `relations` option — declared at query time; most common pattern.
- `QueryBuilder` joins — full control; covered in Section 12.

6.1 Eager Loading — `eager: true`

Setting `eager: true` on a relation tells TypeORM to always load that related entity whenever the owner is queried — no matter how the query is made.

src/entity/User.ts (excerpt)

```
@OneToOne(() => Profile, { nullable: true, cascade: true, eager: true })
@JoinColumn()
profile: Profile | null;
```

src/index.ts (usage)

```
import "reflect-metadata";
import { AppDataSource } from "../data-source";
import { User } from "../entity/User";

AppDataSource.initialize().then(async () => {
  const userRepo = AppDataSource.getRepository(User);

  // No "relations" option needed – profile is always loaded
  const user = await userRepo.findOneBy({ id: 1 });

  console.log(user?.profile); // Profile entity is present automatically
});
```

Output:

```
SQL executed:
SELECT user.*, profile.*
FROM users user
LEFT JOIN profiles profile ON profile.id = user.profileId
WHERE user.id = 1;
```

Console:

```
Profile { id: 1, bio: 'Node.js developer', website: 'https://example.com' }
```

Important — use eager sparingly: Eager loading adds a JOIN to every query on that entity — even when you do not need the related data. This creates unnecessary database load. Reserve `eager: true` for small, always-needed relations (like a profile on a user). For larger relations (like all products in a category), load explicitly at query time instead.

6.2 Explicit `relations` Option — The Recommended Approach

STAGE 1 Load related entities on demand by passing a `relations` array to `find` / `findOne`. This gives you precise control over which relations are loaded in each query.

src/index.ts (loading relations explicitly)

```
import "reflect-metadata";
import { AppDataSource } from "../data-source";
import { Product } from "../entity/Product";

AppDataSource.initialize().then(async () => {
  const productRepo = AppDataSource.getRepository(Product);

  // Load a product WITH its category and tags
  const product = await productRepo.findOne({
    where: { id: 1 },
    relations: { category: true, tags: true },
  });

  console.log("Category:", product?.category?.name);
  console.log("Tags:", product?.tags.map(t => t.name));

  // Load a product WITHOUT relations (just the product columns)
  const bareProduct = await productRepo.findOneBy({ id: 1 });
  console.log("Category on bare load:", bareProduct?.category); // undefined
});
```

Output:

```
SQL for the first query (with relations):
SELECT product.*, category.*, tag.*
FROM   products product
LEFT JOIN categories category ON category.id = product.categoryId
LEFT JOIN product_tags pt     ON pt.productId = product.id
LEFT JOIN tags tag           ON tag.id = pt.tagId
WHERE  product.id = 1;
```



```
Console:
Category: Electronics
Tags: [ 'wireless', 'portable' ]
Category on bare load: undefined
```

6.3 The N+1 Query Problem

The N+1 problem occurs when you load a list of entities and then access a relation on each one in a loop — triggering one query for the list and then one additional query per entity for the related data.

src/index.ts – N+1 antipattern

```
AppDataSource.initialize().then(async () => {
  const productRepo = AppDataSource.getRepository(Product);

  // Query 1: fetch all products (no relations)
  const products = await productRepo.find();

  for (const product of products) {
    // Query 2, 3, 4 ... N+1: fetches category for EACH product separately
    // This happens implicitly if lazy loading is used, or explicitly if you call findOneBy
    here
    console.log(product.category?.name); // undefined – relation was not loaded!
  }
});
```

Output:

Without relations loaded: product.category is **undefined** on every iteration.
If you try to load it in the loop with a separate query per product (N products = N+1 queries total), performance degrades linearly with dataset size.

The fix is to request the relation in the initial `find()` call — TypeORM uses a single JOIN rather than N additional queries:

src/index.ts – correct approach

```
AppDataSource.initialize().then(async () => {
  const productRepo = AppDataSource.getRepository(Product);

  // Single query with JOIN – relations loaded in one round trip
```

```
const products = await productRepo.find({
  relations: { category: true, tags: true },
});

for (const product of products) {
  console.log(`${product.name} - ${product.category?.name ?? "uncategorised"}`);
}
});
```

Output:

SQL: Single SELECT with LEFT JOINS – one database round trip.

Wireless Headphones – Electronics
Leather Wallet – Accessories
Running Shoes – uncategorised

Key Insight: Always load what you need in the initial query. The `relations` option is the idiomatic TypeORM way to do this for simple cases. For complex filtering, ordering, or partial-column selection on related entities, use `QueryBuilder` (covered in Section 12).

7. Self-Referential Relations

A table can reference itself — the most common case being a hierarchical or nested structure, such as a category with subcategories, or a comment with replies.

src/entity/Category.ts (self-referential)

```
import {
  Entity, PrimaryGeneratedColumn, Column,
  OneToMany, ManyToOne,
} from "typeorm";

@Entity("categories")
export class Category {
  @PrimaryGeneratedColumn()
  id: number;
```

```
@Column({ length: 80 })
name: string;

// Parent category – a subcategory has a parent; top-level categories have none
@ManyToOne(() => Category, (category) => category.children, { nullable: true })
parent: Category | null;

// Child categories of this one
@OneToMany(() => Category, (category) => category.parent)
children: Category[];
}
```

Output:

Generated SQL:

```
CREATE TABLE categories (
  "id"      integer PRIMARY KEY AUTOINCREMENT NOT NULL,
  "name"    varchar(80) NOT NULL,
  "parentId" integer,
  FOREIGN KEY ("parentId") REFERENCES "categories" ("id") ON DELETE NO ACTION
);
```

The entity references itself: `() => Category` points to the same class. TypeORM handles the circular reference correctly because the type function is evaluated lazily.

8. Consolidated Entity Files for This Subtopic

The following shows the final state of all entity files after adding relations. Register them all in `data-source.ts`.

```
src/
├─ entity/
│   ├─ Category.ts      ← OneToMany → Product, self-referential parent/children
│   ├─ Product.ts       ← ManyToOne → Category, ManyToMany ↔ Tag
│   ├─ Profile.ts       ← OneToOne ↔ User (inverse)
│   ├─ ProductStatus.ts ← enum (moved to own file)
│   └─ Tag.ts           ← ManyToMany ↔ Product (inverse)
```

```

|   └─ User.ts           ← OneToOne → Profile (owner)
|   └─ data-source.ts
|   └─ index.ts

```

src/data-source.ts

```

import "reflect-metadata";
import { DataSource } from "typeorm";
import { User }      from "../entity/User";
import { Profile }   from "../entity/Profile";
import { Category }  from "../entity/Category";
import { Product }   from "../entity/Product";
import { Tag }       from "../entity/Tag";

export const AppDataSource = new DataSource({
  type: "sqlite",
  database: "dev.db",
  synchronize: true,
  logging: true,
  entities: [User, Profile, Category, Product, Tag],
});

```

Output:

On initialisation, TypeORM creates (or verifies) these tables in order:
 profiles, users, categories, tags, products, product_tags

Summary

Concept	Key Point
@OneToOne	One row in A maps to at most one row in B. The owning side is marked with @JoinColumn() , which creates the FK column.
@ManyToOne / @OneToMany	

Concept	Key Point
	The Many side is always the owning side and holds the FK column. No <code>@JoinColumn</code> needed — TypeORM infers the FK from <code>@ManyToOne</code> .
<code>@ManyToMany</code>	Requires a junction table. The owning side is marked with <code>@JoinTable()</code> . TypeORM creates and manages the junction table automatically.
<code>@JoinColumn</code>	Required on the owning side of One-to-One relations. Specifies which entity holds the FK column. Accepts a <code>name</code> option to override the default column name.
<code>@JoinTable</code>	Required on the owning side of Many-to-Many relations. Defines the junction table name and FK column names.
Inverse side	The second argument to every relation decorator — a function pointing to the other entity's inverse property. Enables bidirectional navigation without adding DB columns.
<code>cascade</code> option	TypeORM-level propagation of save/remove operations. Distinct from database-level <code>onDelete</code> . Use <code>true</code> or an array of specific operation strings.
<code>onDelete</code>	Database-level referential action (<code>"CASCADE"</code> , <code>"SET NULL"</code> , <code>"NO ACTION"</code>). Enforced by the RDBMS regardless of how the delete occurs.
Eager loading	<code>eager: true</code> on a relation always loads the related entity. Convenient for small, always-needed relations; avoid on large or rarely-needed ones.
Explicit <code>relations</code> option	

Concept	Key Point
	Pass <code>{ relations: { propertyName: true } }</code> to <code>find / findOne</code> . Recommended default — precise and avoids unnecessary JOINS.
N+1 query problem	Accessing a relation inside a loop triggers one query per iteration. Prevent it by loading all needed relations in the initial query.
Self-referential relation	An entity can reference itself for hierarchical structures. Both sides of the relation point to the same entity class.

Interview Questions

1. On which side of a One-to-One relation does the foreign key column live, and how does TypeORM know which side that is?

Expected answer: The FK column lives on whichever side is decorated with `@JoinColumn()` — the owning side. TypeORM requires exactly one side to carry `@JoinColumn` for a One-to-One relation; without it, TypeORM throws a configuration error at initialisation time. The `@JoinColumn` decorator generates the FK column in that entity's table.

2. In a One-to-Many / Many-to-One relation, which entity holds the foreign key and why?

Expected answer: The Many side always holds the FK — it is the only arrangement that avoids redundancy. If the One side held the FK, a category with ten products would need to store ten FK values, which would violate first normal form. The Many side (e.g. `Product`) stores a single `categoryId`, and every product row has its own copy of that value pointing to exactly one category row.

3. Why does a Many-to-Many relation require a junction table, and what does TypeORM generate for it?

Expected answer: A single FK column on either entity can only store one reference — it cannot represent a many-to-many pairing. A junction table resolves this by storing pairs of FK values,

one from each entity, with the composite of both as the primary key. TypeORM generates the junction table from the `@JoinTable` configuration, creating it with both FK columns and a composite PK. TypeORM also inserts and deletes rows in the junction table automatically when you add or remove entries from the relation array.

4. What is the difference between `cascade: true` on a TypeORM relation and `onDelete: "CASCADE"` ?

Expected answer: `cascade: true` is TypeORM application-level behaviour — it instructs TypeORM's JavaScript runtime to automatically call `save()` or `remove()` on related entities when the owner is persisted or removed via TypeORM. `onDelete: "CASCADE"` is a SQL constraint embedded in the table DDL — the database enforces it regardless of how the delete happens (TypeORM, raw SQL, another service). Both can and often should be used together.

5. What is the N+1 query problem and how do you prevent it in TypeORM?

Expected answer: N+1 occurs when you load a list of N entities and then access an unloaded relation on each in a loop, causing N additional queries — one per entity. The total is N+1 database round trips. Prevent it by loading the needed relations in the initial query using the `relations` option in `find()` / `findOne()`, or by using QueryBuilder with explicit joins. TypeORM then issues a single query with JOINS rather than separate queries per entity.

6. When would you use `eager: true` on a relation, and what are its drawbacks?

Expected answer: Eager loading is appropriate for relations that are small in size and needed in virtually every query on that entity — a User's Profile is a classic example. The drawback is that it adds a JOIN to every query automatically, even when the caller does not need the related data. This wastes database I/O and can hide performance problems. The recommended approach is to keep relations lazy by default and request them explicitly at query time with the `relations` option.

7. You have a `Category` with a `@OneToMany` to `Product`, and `onDelete: "CASCADE"` is set on the `Product.category @ManyToOne`. A junior developer deletes a category via raw SQL — do the products get deleted?

Expected answer: Yes. `onDelete: "CASCADE"` is a database-level constraint. It is enforced by the RDBMS for any delete on the `categories` table — regardless of whether it came from TypeORM, raw SQL, or a database management tool. TypeORM's application-level `cascade` option would not fire (since TypeORM was not involved), but the database constraint is absolute.

8. How would you model a self-referential parent/child hierarchy (e.g. categories with subcategories) using TypeORM decorators?

Expected answer: Use `@ManyToOne(() => Category, cat => cat.children, { nullable: true })` for the `parent` property (nullable so top-level categories have no parent), and `@OneToMany(() => Category, cat => cat.parent)` for the `children` property. Both sides reference the same entity class. TypeORM generates a single `parentId` FK column on the categories table that references the same table's `id` column.

Practical Assignment — Wiring Up a Product Catalogue with Relations

Objective: Extend the product catalogue project from 11-01 to include four related entities with One-to-One, One-to-Many, and Many-to-Many relations. Verify the generated schema, then write code to create and retrieve related data.

Required Entities and Relations

- **User** — `@OneToOne` (owning) to **Profile**; cascade insert + update.
- **Profile** — inverse `@OneToOne` back to **User**.
- **Category** — `@OneToMany` to **Product**; self-referential parent/children.
- **Product** — `@ManyToOne` to **Category** (`onDelete: "SET NULL"`); `@ManyToMany` (owning) to **Tag** with cascade.
- **Tag** — inverse `@ManyToMany` back to **Product**.

Step-by-step Instructions

1. Create all entity files as described in Section 8 of these notes. Register all five entities in `data-source.ts`.
2. In `src/index.ts`, initialise the DataSource, then write code to create and save the following using repositories:
 - A `Category` named "Electronics" with slug "electronics".
 - Two `Tag` s: "wireless" and "portable".

- A Product named "Wireless Headphones" , price 79.99 , status Active , linked to the Electronics category and both tags.
- A User with email "alice@example.com" and a Profile with bio "Node.js developer" .

3. After saving, query back the product with all relations loaded:

```
const product = await productRepo.findOne({
  where: { name: "Wireless Headphones" },
  relations: { category: true, tags: true },
});
```

4. Log the product name, category name, and tag names to the console.

5. Run with `npm run ts-node src/index.ts` and confirm the output matches the expected output below.

Expected Output

Output:

```
query: INSERT INTO "categories" ("name", "slug") VALUES (?, ?)
query: INSERT INTO "tags" ("name") VALUES (?)
query: INSERT INTO "tags" ("name") VALUES (?)
query: INSERT INTO "products" ("name", "price", "status", "categoryId") VALUES (?, ?, ?, ?)
query: INSERT INTO "product_tags" ("productId", "tagId") VALUES (?, ?), (?, ?)
query: INSERT INTO "profiles" ("bio") VALUES (?)
query: INSERT INTO "users" ("email", "passwordHash", "profileId") VALUES (?, ?, ?)
```

```
Product: Wireless Headphones
Category: Electronics
Tags: wireless, portable
```

Bonus Challenge

Add a subcategory "Headphones" whose parent is the "Electronics" category. Move the Wireless Headphones product to the "Headphones" subcategory. Then query all categories and load their children and products relations. Log the hierarchy to the console in the format:

Output:

Electronics

└ Headphones

└ Wireless Headphones (\$79.99)

Module 4: Node.js, Express & TypeORM — Section 11, Subtopic 02 — Relations

Next: 11-03 **CRUD with TypeORM** — Repositories, Entity Manager, save / find / update / remove patterns and custom repositories

